Lab: Operation New Crew

Modules Covered: M08 – M09 **Duration:** 90 – 120 minutes **Difficulty:** Beginner-Intermediate

Prerequisites: A fresh Ubuntu VM (local or cloud) with sudo access

---

The Scenario

Two weeks after the Lockdown incident, TechNova Corp is growing. You rebuilt the staging environment, locked down permissions, and wrote the security policy. Management noticed. Now they're hiring — and they want it done right this time.

Your team lead Anas messages you:

> "Good news — we're expanding the team. Five new hires start Monday. I need you to get them set up before they arrive. Here's the list:
>
> - **Hammam, Muath, Saad** — developers
> - **Maitha** — sysadmin
> - **Renad** — deployer
>
> After the Faisal incident, we're not doing shared accounts ever again. Each person gets their own account, proper group membership, the right level of sudo access, and SSH key authentication. No shortcuts. No 'we'll fix it later.'
>
> You have the rest of the day. Set it up so that when they sit down Monday morning, **everything just works.**"

You open your terminal. Five accounts, three roles, zero room for error.

Before You Start

This lab requires **no setup script**. Everything you do uses real Linux commands on a stock Ubuntu VM. You'll create users, manage groups, configure sudo, and set up SSH keys.

Rules:

- Type every command yourself — no copy-paste
- When you see **"Try this (it will fail)"**, do it anyway — the error IS the lesson
- When you see a checkbox, pause and make sure you understand before moving on

---

Part 1: Who's Already Here? (M08)

Before you add anyone new, you need to understand who's already on this system and how Linux tracks users.

Task 1.1 — Who are you?

Confirm your identity on this system:

```
whoami
```

Now get the full picture — your UID, GID, and every group you belong to:

```
id
```

Write down your UID. You'll compare it to the new users' UIDs later.

Task 1.2 — What groups do you belong to?

Display your group memberships by name:

```
groups
```

Compare this to the groups listed in the `id` output. Same groups, different format — `groups` gives you just the names, `id` gives you the numbers too.

Task 1.3 — Read the user database

Every user on this system is listed in `/etc/passwd`:

```
cat /etc/passwd
```

That's a lot of users — most are system accounts. Each line follows this format:

```
username:x:UID:GID:comment:home_directory:shell
```

The `x` in the password field means the actual hash is stored elsewhere (in `/etc/shadow`).

Filter to just the human users — those with home directories under `/home`:

```
grep ":/home/" /etc/passwd
```

> **Tip:** System users have UIDs 0–999. Regular (human) users start at UID 1000. The system creates service accounts for things like `www-data`, `sshd`, and `nobody` — you should never log in as these.

Task 1.4 — Try to read the password hashes (it will fail)

```
cat /etc/shadow
```

You should see `Permission denied`. Now try with elevated privileges:

```
sudo cat /etc/shadow
```

This separation IS the security model — usernames in `/etc/passwd` are readable by everyone (programs need them), but password hashes in `/etc/shadow` are protected. Only root can read them.

Look at the hash format. If you see a line with `$6$` at the start of the hash, that means SHA-512. The `$y$` prefix means yescrypt (newer Ubuntu default). Either way — these are one-way hashes, not reversible passwords.

Task 1.5 — Read the group database

Every group on this system is listed in `/etc/group`:

```
cat /etc/group
```

Each line follows this format:

```
group_name:x:GID:member1,member2
```

Now check who has sudo access on this machine:

```
getent group sudo
```

Groups are how Linux organizes permissions at scale. Instead of granting access to each user individually, you put users in groups and grant access to the group.

Task 1.6 — Who else is here?

Check who's currently logged into this system:

```
w
```

This shows every active session — who they are, where they connected from, how long they've been idle, and what they're running.

Now check the last 5 login sessions:

```
last -5
```

On shared servers, this is how you check who's been active. After the Faisal incident, this is the kind of audit trail that would have told the team exactly who logged in and when.

Checkpoint: Part 1

- [ ] You know your username, UID, GID, and group memberships
- [ ] You can read /etc/passwd and identify system vs regular users
- [ ] You understand why /etc/shadow requires sudo
- [ ] You can check who's currently logged in with `w` and `last`

Part 2: Onboard the New Crew (M08)

You know who's on the system. Now it's time to create accounts for the new team and organize them into groups.

Task 2.1 — Create the groups

Before creating any users, set up the organizational structure. Groups first, people second:

```
sudo groupadd developers
sudo groupadd sysadmins
sudo groupadd deployers
```

Verify the groups were created:

```
cat /etc/group | tail -5
```

You should see all three new groups at the bottom of the file. Creating the structure before the users means you can assign group membership during or immediately after account creation.

Task 2.2 — Create the first user interactively

Start with Hammam using the friendly, interactive tool:

```
sudo adduser hammam
```

It will prompt you for:

- A password (type it twice — nothing shows on screen, that's normal)
- Full name and optional info (you can press Enter to skip these)

Verify the account was created:

```
id hammam
```

You should see a UID (probably 1001), a primary group `hammam`, and no secondary groups yet.

> **Tip:** `adduser` is the high-level, interactive way to create users. It creates the home directory, copies skeleton files, sets the shell, and prompts for a password — all in one step. Good for one-off setup when you're at the keyboard.

Task 2.3 — Create the rest with useradd (script-style)

For the remaining four users, use the low-level command:

```
sudo useradd -m -s /bin/bash muath
sudo useradd -m -s /bin/bash saad
sudo useradd -m -s /bin/bash maitha
sudo useradd -m -s /bin/bash renad
```

The flags: `-m` creates a home directory, `-s /bin/bash` sets their default shell. Without `-m`, the user would have no home directory. Without `-s`, they might get `/bin/sh` or no shell at all.

Now set passwords for each user:

```
sudo passwd muath
sudo passwd saad
sudo passwd maitha
sudo passwd renad
```

> **Tip:** `adduser` is interactive and does everything for you — home directory, shell, password, all in one step. `useradd` is the low-level tool that does exactly what you tell it and nothing more. Scripts use `useradd` because automation can't answer interactive prompts.

Task 2.4 — Add users to their groups

Assign each user to the correct team:

```
sudo usermod -aG developers hammam
sudo usermod -aG developers muath
sudo usermod -aG developers saad
sudo usermod -aG sysadmins maitha
sudo usermod -aG deployers renad
```

> **WARNING:** The `-aG` flags are critical. The `-G` flag sets supplementary groups, and `-a` means **append**. If you use `-G` without `-a`, it **replaces** all existing supplementary group memberships. Run `sudo usermod -G developers hammam` and he loses membership in every other group he belonged to. This mistake has ended careers. Always use `-aG` together.

Task 2.5 — Verify all group memberships

Check each group to confirm the right people are in the right teams:

```
getent group developers
getent group sysadmins
getent group deployers
```

Expected output:

- `developers` should show `hammam,muath,saad`
- `sysadmins` should show `maitha`
- `deployers` should show `renad`

Now check the full picture for one user:

```
id hammam
```

You should see his UID, his primary group (`hammam`), and `developers` as a supplementary group.

Task 2.6 — Switch to a new user (it will fail first)

Try to switch to Hammam's account:

```
su hammam
```

It asks for Hammam's password. If you don't remember it, you're stuck. This is how `su` works — it requires the **target** user's password.

Instead, use your sudo privileges to switch:

```
sudo su - hammam
```

Now verify you're actually Hammam:

```
whoami
pwd
id
```

You should see `hammam`, a home directory of `/home/hammam`, and his UID and groups.

Return to your own account:

```
exit
```

> **Tip:** `su` requires the target user's password. `sudo su -` uses YOUR sudo privileges to switch. The `-` gives a full login shell — it loads the target user's `.bashrc`, sets their home directory, and resets environment variables. Without the `-`, you'd keep your own environment, which leads to confusing behavior.

Task 2.7 — Create a shared project directory

The developers need a shared workspace. Create it and set ownership:

```
sudo mkdir -p /var/www/technova-project
sudo chown -R :developers /var/www/technova-project
sudo chmod -R 770 /var/www/technova-project
sudo chmod g+s /var/www/technova-project
```

The `g+s` flag sets the **setgid bit** on the directory. This means any new file created inside will automatically inherit the `developers` group — so when Hammam creates a file, Muath and Saad can access it immediately without anyone running `chown` manually.

Test it:

```
sudo su - hammam
touch /var/www/technova-project/test-file.txt
ls -l /var/www/technova-project/test-file.txt
exit
```

The file should show `developers` as the group owner, not `hammam`. That's setgid doing its job.

Task 2.8 — List all human users on the system

See every human user account on this machine:

```
awk -F: '$3 ≥ 1000 && $3 < 65534 {print $1, $3}' /etc/passwd
```

You should see your original user plus all five new accounts. This works by filtering `/etc/passwd` by UID range — UIDs 1000+ are human users, and 65534 is the `nobody` account (excluded).

Checkpoint: Part 2

- [ ] You created three groups: developers, sysadmins, deployers
- [ ] You created five user accounts using both adduser and useradd
- [ ] You added each user to the correct group with `usermod -aG`
- [ ] You know the difference between `su` and `sudo su -`
- [ ] You set up a shared directory with setgid for automatic group inheritance

---

Part 3: The Right Level of Power (M08)

Your team has accounts and groups. But right now, none of them can install software, restart services, or do anything administrative. That's the default — and it's correct. Now you'll grant each role exactly the power they need. No more, no less.

Task 3.1 — Check who has sudo access

See which users currently belong to the `sudo` group:

```
getent group sudo
```

Now list your own sudo privileges:

```
sudo -l
```

Only your original account should have sudo. The five new accounts have nothing — they can log in, edit files in their home directory, and run non-privileged commands. That's it.

Task 3.2 — Give Maitha full sudo access

Maitha is the sysadmin — she needs full administrative access to manage the system:

```
sudo usermod -aG sudo maitha
```

Verify she's now in the `sudo` group:

```
id maitha
```

You should see `sudo` listed among her groups. Adding a user to the `sudo` group is the standard Ubuntu way to grant full admin access. No file editing, no special syntax — just group membership. The `/etc/sudoers` file already contains a rule that gives the `sudo` group full privileges.

Task 3.3 — Create a targeted sudo rule for Renad

Renad is the deployer — she needs to copy files to the project directory and restart nginx, nothing else. You'll create a scoped rule using a drop-in file:

```
sudo visudo -f /etc/sudoers.d/deployers
```

Add the following lines in the editor:

```
# Deployer role - copy files and restart services only
%deployers ALL=(ALL) NOPASSWD: /usr/bin/cp /tmp/app/* /var/www/technova-pro
%deployers ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx
%deployers ALL=(ALL) NOPASSWD: /usr/bin/systemctl status nginx
```

Save and exit. Let's break down the syntax:

- `%deployers` — the `%` prefix means this applies to a **group**, not a single user
- `ALL=` — on any host (matters for multi-server setups, but required syntax)
- `(ALL)` — run the command as any user (typically root)
- `NOPASSWD:` — don't prompt for a password when running this command
- The rest is the exact command path that's allowed

Two things to understand about this approach:

1. `visudo` **validates syntax before saving.** A typo in a sudoers file can lock everyone out of sudo entirely. Never edit `/etc/sudoers` or its drop-in files with a regular text editor.
2. `/etc/sudoers.d/` **keeps rules organized.** One file per role — easy to audit, easy to revoke. To remove deployer privileges later, just delete the file.

Task 3.4 — Test Maitha's full sudo

Switch to Maitha and try an administrative command:

```
sudo su - maitha
sudo apt update
```

It should work — Maitha has full sudo access through the `sudo` group. She can install packages, restart services, read logs, modify system files — everything an administrator needs.

Return to your account:

```
exit
```

Task 3.5 — Test Hammam's access (it will fail)

Switch to Hammam and try an administrative command:

```
sudo su - hammam
sudo apt install tree
```

You should see:

```
hammam is not in the sudoers file. This incident will be reported.
```

That last line sounds ominous, and it's intentional. Every failed sudo attempt is logged to `/var/log/auth.log`. On a production server, security monitoring tools watch this file for repeated failures — it's one of the first signs of an attacker probing for privilege escalation.

Return to your account:

```
exit
```

This IS the principle of least privilege working correctly. Developers write code — they don't administer the server. If Hammam's account is compromised, the attacker can't install malware, modify system files, or pivot to other services. The blast radius is contained to Hammam's home directory and the shared project folder.

Task 3.6 — Test Renad's limited sudo

Switch to Renad and test a command she's allowed to run:

```
sudo su - renad
sudo systemctl status nginx
```

The sudo part should succeed (nginx may or may not be installed — what matters is that sudo doesn't reject the command).

Now try something she's NOT allowed to do:

```
sudo apt update
```

This should be denied. Renad can only run the specific commands listed in `/etc/sudoers.d/deployers`. She can deploy (copy files to the project directory, restart nginx) but she can't install packages, read system logs, or modify anything else.

Return to your account:

```
exit
```

If Renad's deploy credentials are compromised, the attacker can overwrite the web application and restart nginx — that's bad, but it's recoverable. They can't install a rootkit, create new user accounts, or read other users' data. Scoped sudo limits the blast radius.

Checkpoint: Part 3

- [ ] You can check who has sudo access with `getent group sudo`
- [ ] You gave the sysadmin full sudo via the `sudo` group
- [ ] You created a targeted sudoers.d rule for the deployer
- [ ] You tested that developers have NO sudo — and understand why
- [ ] You understand the Linux-to-cloud parallel: users = IAM users, groups = IAM groups, sudo rules = IAM policies

> The principle of least privilege isn't a suggestion — it's the foundation of every security model from Linux to AWS IAM to Google Cloud IAM. You just implemented it with six commands.

Part 4: Lock the Door (M09)

Your team has accounts and the right level of power. But they're still logging in with passwords — and every password is a target. Time to switch to SSH keys and make brute-force attacks mathematically impossible.

Task 4.1 — Make sure SSH is running

Check whether the SSH service is active on your machine:

```
systemctl status sshd
```

On some Ubuntu installations, the service is named `ssh` instead of `sshd`. If neither is found, install it:

```
sudo apt install openssh-server -y
```

If installed but not running, start it:

```
sudo systemctl start sshd
```

SSH (Secure Shell) is how you connect to remote servers. Even though you're already on this machine, SSH can also connect to `localhost` — which makes it perfect for testing key-based authentication without a second server.

Task 4.2 — Generate your SSH key pair

Generate an Ed25519 key pair for your account:

```
ssh-keygen -t ed25519 -C "$USER@technova"
```

The prompts:

- **File location:** accept the default (`~/.ssh/id_ed25519`) — just press Enter
- **Passphrase:** add one — this protects the key if your laptop is stolen

Ed25519 is modern, fast, and secure. RSA 4096 is the alternative for older systems that don't support Ed25519. Ed25519 keys are shorter but equally strong.

What just happened: you created TWO files — a **private key** (your identity, NEVER share) and a **public key** (a padlock you give to servers).

Task 4.3 — Examine the key files

List the contents of your `.ssh` directory:

```
ls -la ~/.ssh/
```

Pay attention to the permissions:

- `id_ed25519` should be `600` (owner read/write only) — if this file is readable by others, SSH refuses to use it
- `id_ed25519.pub` should be `644` (anyone can read) — this is safe to share

View the public key:

```
cat ~/.ssh/id_ed25519.pub
```

It's one long line starting with `ssh-ed25519`. This is what goes on servers. The private key stays on your machine. Always.

Task 4.4 — Install your key on localhost

Copy your public key to the local machine's authorized keys:

```
ssh-copy-id localhost
```

You'll be asked for your password ONE LAST TIME. After this, keys take over.

What `ssh-copy-id` did: it appended your public key to `~/.ssh/authorized_keys` on the target (localhost in this case). That file is the server's list of “keys I trust.”

Task 4.5 — Test passwordless SSH

Connect to localhost via SSH:

```
ssh localhost
```

No password prompt. Verify you're in a new SSH session:

```
whoami
hostname
exit
```

The `exit` brings you back to your original session.

Here's what happened behind the scenes: the server sent a challenge, your private key signed it, the server verified the signature with your public key. No password crossed the network.

Task 4.6 — Create an SSH config shortcut

Open (or create) your SSH config file:

```
nano ~/.ssh/config
```

Add the following:

```
Host technova-local
    HostName localhost
    User <your-username>
    IdentityFile ~/.ssh/id_ed25519
```

Replace `<your-username>` with your actual username.

Set the correct permissions on the config file:

```
chmod 600 ~/.ssh/config
```

Test the shortcut:

```
ssh technova-local
```

Then `exit`.

Instead of remembering `ssh -i ~/.ssh/id_ed25519 anas@192.168.1.50`, you just type `ssh technova-local`. In real life, you'll have entries for dev, staging, production, jump boxes — one file that maps friendly names to connection details.

Task 4.7 — Set up SSH keys for Maitha

Switch to Maitha and generate her key pair:

```
sudo su - maitha
ssh-keygen -t ed25519 -C "maitha@technova"
```

Accept defaults, add a passphrase.

Copy her key to localhost:

```
ssh-copy-id localhost
```

Test:

```
ssh localhost
whoami
exit
```

Return to your account:

```
exit
```

This is the workflow for every new team member: generate key, copy to server, test. Repeat for each server they need access to. In real companies, this process is often automated with configuration management tools like Ansible.

Checkpoint: Part 4

- [ ] You generated an Ed25519 SSH key pair
- [ ] You understand the difference between private key (600, never share) and public key (644, give to servers)
- [ ] You installed your key with ssh-copy-id and logged in without a password
- [ ] You created an SSH config shortcut
- [ ] You set up keys for another user (Maitha)

Part 5: Harden the Server (M09)

SSH keys work. But the server still accepts password logins too — which means bots can still try 10,000 passwords a day. Time to shut that door completely and set the server's identity.

Task 5.1 — Read the current SSH server config

Strip out comments and empty lines to see what's actually configured:

```
sudo cat /etc/ssh/sshd_config | grep -v '^#' | grep -v '^$'
```

Look at the output and identify: Is password authentication enabled? Is root login allowed? What port is SSH on?

This file controls how the SSH **server** behaves — who can connect, how they authenticate. The client config (`~/.ssh/config`) controls how **you** connect to others. Don't confuse them.

Task 5.2 — CRITICAL: Open a second terminal (safety net)

Before changing anything, open a **second terminal** or SSH session to this machine. Keep it open.

Why: if your SSH config changes break authentication, your existing session stays alive — it's your lifeline. If you close your only session and keys don't work and passwords are disabled... you're locked out. Recovery means console access or rebuilding.

This is not optional. Open the second terminal now.

Task 5.3 — Edit the SSH server config

```
sudo nano /etc/ssh/sshd_config
```

Find and modify these settings (some may be commented out — uncomment them):

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication yes    # Keep 'yes' for now - we'll test before disa
```

What each setting does:

- `PermitRootLogin no` — nobody can SSH directly as root. Even if they have root's password, the door is closed. Admin work goes through sudo.
- `PubkeyAuthentication yes` — enables SSH key authentication (usually already yes).
- `PasswordAuthentication yes` — we're keeping this on temporarily so we can test keys first. Disabling it prematurely is the #1 way people lock themselves out.

Save and exit.

Task 5.4 — Validate and apply the config

Test the config syntax BEFORE restarting:

```
sudo sshd -t
```

If you see no output, the config is valid. If you see errors, go back and fix them.

Apply the changes:

```
sudo systemctl restart sshd
```

Now test SSH in your SECOND terminal:

```
ssh localhost
```

If it works with keys (no password prompt), your config is correct.

Task 5.5 — Set the server hostname

View the current hostname:

```
hostname
hostnamectl
```

Set a meaningful hostname:

```
sudo hostnamectl set-hostname technova-staging
```

Verify:

```
hostnamectl
```

Hostnames identify servers. In a fleet, you'll have names like `web-prod-01`, `db-staging-02`, `cache-us-east-1`. A server named `ubuntu` or `localhost` tells you nothing. In cloud providers (AWS, GCP), hostnames are often set from instance metadata automatically.

> **Tip:** The prompt won't update until you open a new shell session.

Task 5.6 — Write the onboarding completion report

Create a document summarizing everything you set up:

```
cat > ~/onboarding-complete.txt << EOF
═══ TEAM ONBOARDING REPORT ═══
Date: $(date)
Prepared by: $USER

--- Users Created ---
hammam  (UID: see id hammam)  - Developer
muath   (UID: see id muath)   - Developer
saad    (UID: see id saad)    - Developer
maitha  (UID: see id maitha)  - Sysadmin
renad   (UID: see id renad)   - Deployer

--- Groups ---
developers: hammam, muath, saad
sysadmins:  maitha
deployers:  renad

--- Sudo Access ---
maitha:  Full sudo (sudo group member)
renad:   Limited sudo (deploy commands only, via /etc/sudoers.d/deployers)
developers: No sudo access

--- SSH ---
Key type: Ed25519
Password auth: Still enabled (disable after all team keys verified)
Root login: Disabled

--- Server ---
Hostname: technova-staging

═══ END OF REPORT ═══
EOF
```

Review it:

```
cat ~/onboarding-complete.txt
```

Documentation isn't exciting but it's how the next person knows what was done. When Anas reviews your work, this report tells him the full picture in 30 seconds.

Checkpoint: Part 5

- [ ] You read and understood the SSH server config (`sshd_config`)
- [ ] You disabled root login and verified key authentication works
- [ ] You validated the config with `sshd -t` before restarting
- [ ] You kept a second session open as a safety net
- [ ] You set a meaningful hostname with `hostnamectl`
- [ ] You documented everything in an onboarding report

Final Verification

Run this block to review everything you built in one shot:

```
echo "═══ Users ═══"
awk -F: '$3 ≥ 1000 && $3 < 65534 {print $1, $3}' /etc/passwd

echo ""
echo "═══ Groups ═══"
getent group developers
getent group sysadmins
getent group deployers

echo ""
echo "═══ Sudo Access ═══"
getent group sudo

echo ""
echo "═══ SSH Keys ═══"
ls -la ~/.ssh/

echo ""
echo "═══ Server Identity ═══"
hostnamectl
```

What You Accomplished

You started with a solo account on a blank server and built a complete team infrastructure:

1. **Assessed the current system** — identified existing users, groups, and access patterns
2. **Created five accounts** — using both adduser and useradd, organized into role-based groups
3. **Configured least-privilege sudo** — full admin for the sysadmin, scoped commands for the deployer, nothing for developers

4. **Set up SSH key authentication** — generated Ed25519 keys, tested passwordless login via localhost
5. **Hardened the server** — disabled root login, set the hostname, documented everything

> The same pattern you just implemented — individual accounts, role-based groups, scoped permissions, key-based authentication — is exactly how AWS IAM, Google Cloud IAM, and Azure AD work. You just learned it on a Linux terminal.

> Next: your team is set up, the server is secure. But it's empty — no web server, no applications, no monitoring. In the next modules, you'll install packages with APT and manage services with systemctl.

Bonus Challenges

Finished early? Try these without looking at the instructions above.

Challenge 1: Find Every Human User

Write a one-liner that lists all users with UID >= 1000, excluding `nobody`.

(This one is easy — it's a repeat from Task 2.8. The challenge is doing it from memory.)

▶ Solution

Challenge 2: Check the Audit Trail

Read the last 20 lines of the real system authentication log to see every sudo attempt (successful and failed). Find the “This incident will be reported” entry from when Hammam tried sudo.

▶ Solution

Challenge 3: Lock and Unlock a User

Lock Hammam's account (simulate him going on vacation), then verify he can't log in, then unlock him.

Hint: `sudo usermod -L hammam`, *then try* `sudo su - hammam`

▶ Solution

Challenge 4: Remove a User Completely

Delete Saad's account AND his home directory (simulate him leaving the company).

Hint: `userdel` *has a flag that removes the home directory too.*

▶ Solution

Challenge 5: Debug an SSH Connection

Use verbose mode to watch the SSH handshake step by step.

Hint: `ssh` *has* `-v`, `-vv`, *and* `-vvv` *flags for increasing verbosity.*

▶ Solution

Command Reference

Commands

Command	What It Does	Module
`whoami`	Display current username	M08
`id`	Show UID, GID, and all groups	M08
`groups`	List group memberships by name	M08
`w`	Show who's logged in and what they're doing	M08
`last`	Show recent login sessions	M08
`adduser`	Create a user interactively	M08
`useradd`	Create a user (low-level, for scripts)	M08
`passwd`	Set or change a user's password	M08
`usermod`	Modify user properties (groups, shell, lock)	M08
`userdel`	Delete a user account	M08
`groupadd`	Create a new group	M08
`gpasswd`	Add or remove group members	M08
`getent`	Query system databases (users, groups)	M08
`sudo`	Run a command with elevated privileges	M08
`visudo`	Safely edit sudoers configuration	M08
`su`	Switch to another user	M08

Command	What It Does	Module
`ssh-keygen`	Generate SSH key pairs	M09
`ssh-copy-id`	Install public key on a server	M09
`ssh`	Connect to a remote server	M09
`hostname`	Display server hostname	M09
`hostnamectl`	View or change server hostname	M09
`sshd -t`	Validate SSH server config syntax	M09
`systemctl`	Manage system services	M09

Key Files

File	Purpose	Module
`/etc/passwd`	User account database (world-readable)	M08
`/etc/shadow`	Password hashes (root-only)	M08
`/etc/group`	Group database	M08
`/etc/sudoers`	Main sudo configuration	M08
`/etc/sudoers.d/`	Drop-in sudo rules (one file per role)	M08
`~/.ssh/id_ed25519`	SSH private key (600, never share)	M09
`~/.ssh/id_ed25519.pub`	SSH public key (644, give to servers)	M09
`~/.ssh/authorized_keys`	Keys allowed to log in (on server)	M09
`~/.ssh/config`	SSH client shortcuts	M09
`/etc/ssh/sshd_config`	SSH server configuration	M09

Lab designed for the Linux Fundamentals for Cloud Computing course.